

- Questions and Answers of FAIKR2

- 1. Introduce the vanilla meta-interpreter, and to explain its clauses.
- 2. Define a meta-interpreter for the Prolog language, where the selection of the subgoal in the current resolvent is right-most rather than left-most (as it is usually).
- 3. Define a meta-interpreter for the Prolog language, where whenever a special predicate `break/0` is encountered, the meta-interpreter directly solve it by asking the user if she wants to continue. The evaluation of subgoal `break/0` will succeed only if the user types "yes". To this end, the candidate can use the predicate `read(Term)`, that "Read the next Prolog term from the current input stream and unify it with Term."
- 4. Present the notion of the cut "!" operator in the Prolog language. Moreover, the candidate should illustrate its usage through a very short example.
- 5. Describe how negation is tackled in Prolog, what is NAF, what is SLDNF, and the issues related with NAF over terms containing unbounded variables.
- 6. Introduce the meta-predicate `clause/2`, showing its use by means of a short example program.
- 7. Describe the (operational) semantics of the predicates `setof`, `bagof`, and `findall`. Moreover, the description should be illustrated by some short examples.
- 8. Describe the distribution semantics adopted in LPAD, using also a short program to illustrate such semantics.
- 9. Introduce the notions of close world assumption and open world assumption, and to briefly discuss how Prolog and Description Logics deal with these aspects.
- 10. Briefly introduce the ALC Description Logics, mentioning the operators that are supported (negation, AND, ALL, EXISTS), and their meaning (possibly with a short example for each operator).
- 11. Within the terminological approach towards the representation of concepts/categories and individuals/instances, the candidate is invited to illustrate the notions of:
 - Disjointness over a set of categories (where are categories)
 - Exhaustive Decomposition of a category into a set of categories.
 - Partition of a category into a set of categories.The candidate is invited to illustrate these notions through a simple example.
- 12. Briefly introduce the notion of Semantic Networks, and to highlight some of the limits that were present in their original formulation.
- 13. Briefly introduce the Knowledge Graph paradigms, and which are the main differences w.r.t. the Semantic Web proposal.
- 14. Briefly introduce the three different approaches (presented in the course), to deal with the reasoning with temporal information.
- 15. Describe the terminology, ontology, and domain-independent axioms of the Event Calculus Framework, explaining its key predicates, and provide a detailed account of the framework axioms.
- 16. Explain Forward reasoning in rule-based systems, and highlight the difference w.r.t. backward reasoning.
- 17. Introduce the RETE algorithm, and to show with a short example in natural language the differences between inter-elements and intra-elements patterns/features.

Questions and Answers of FAIKR2

by [Lorenzo Pellegrino](#) and [Giorgio Scavello](#)

1. Introduce the vanilla meta-interpreter, and to explain its clauses.

The vanilla meta-interpreter is defined as:

```
% 1st clause: tautology
solve(true) :- !.           % It is always satisfied

% 2nd clause: conjunction
solve((A,B)) :-
    !,                     % Not to fall into 3rd clause
    solve(A), solve(B).    % So to solve the conjunction in left-most order

% 3rd clause: atom
solve(A) :-
    clause(A,B),           % Solve the body of A to solve A
    solve(B).
```

2. Define a meta-interpreter for the Prolog language, where the selection of the subgoal in the current resolvent is right-most rather than left-most (as it is usually).

```
solve(true) :- !.
solve((A, B)) :- !, solve(B), solve(A).
solve(A) :- clause(A, B), solve(B).
```

3. Define a meta-interpreter for the Prolog language, where whenever a special predicate break/0 is encountered, the meta-interpreter directly solve it by asking the user if she wants to continue. The evaluation of subgoal break/0 will succeed only if the user types "yes". To this end, the candidate can use the predicate read(Term), that "Read the next Prolog

term from the current input stream and unify it with Term."

```
solve(true) :- !.  
solve((A, B)) :- !, solve(A), solve(B).  
solve(break) :-  
    !, % Not to make it be considered as a normal atom  
    format("Do you want to continue? (yes/no)~n"),  
    read(yes). % success only if the Answer is "yes"  
solve(A) :-  
    clause(A, B),  
    solve(B).
```

4. Present the notion of the cut "!" operator in the Prolog language. Moreover, the candidate should illustrate its usage through a very short example.

choice points: list of all the predicates that can solve a goal by matching it

The cut operator is a goal which always succeeds but cannot be backtracked. Cuts can prevent unwanted backtracking, making the program more efficient. In order to avoid backtracking, the cut tells Prolog to discard all choice points created by goals before the cut in the current clause. In this way if Prolog encounters failure after the cut, it will not revisit any choice points created before the cut within the same clause. Instead, it will fail the entire clause and proceed to the next one, if available.

This behaviour allows us to represent easily the "if-then-else" statement, given

if A then B else C

We can implement it in Prolog by:

```
% Implementation  
if_then_else(Condition, Then, Else) :-  
    Condition,  
    !,  
    Then.  
if_then_else(_,_,Else) :- Else.  
  
% Translation  
if_then_else(A,B,C).
```

5. Describe how negation is tackled in Prolog, what is NAF, what is SLDNF, and the issues related with NAF over terms containing unbounded variables.

In Prolog, the negation of a goal succeeds if and only if the goal can't be proven (it's false) in a finite number of steps. This approach is called NAF (Negation as Failure) and it's incorporated into the Prolog reasoning framework which is SLD (depth-first and left-most) and becomes SLDNF (Selective Linear Definite clause resolution with Negation as Failure).

In SLDNF we'll have that when Prolog faces a literal L we can have 2 cases:

1. Literal is positive ($L \equiv X$) : classical SLD will be applied on the literal.
2. Literal is negative ($L \equiv \neg X$) : classical SLD will be applied on X , if it fails then L is true, otherwise it is false.

The logical problem of NaF is about how quantifiers are used because when we query `capital(X)` we (and Prolog) mean $\exists x : \text{capital}(x)$? When we query `\+capital(X)`, in the same way we mean $\exists x : \neg \text{capital}(x)$? But, from the Prolog point of view, when we query `\+capital(X)` we are negating the goal and so we actually mean $\neg(\exists x : \text{capital}(x)) \equiv (\forall x : \neg \text{capital}(x))$ by De Morgan rule. So prolog should fail if there is only one city that is a capital but instead we wanted it to success and give us that unique name.

For example if there are both capitals and not capitals we obtain

```
city(rome, 3).
city(tokyo, 6).
city(bolo, 1).
city(modena, 1).

capital(X) :-
    city(X, P), P>2.

?- \+capital(X).
false.
```

Here we would have expected `true. X=bolo; X=modena` instead of false. If we take away the capitals then we have

```
% city(rome, 3).
% city(tokyo, 6).
city(bolo, 1).
city(modena, 1).

capital(X) :-
    city(X, P), P>2.

?- \+capital(X).
true.
```

Also here we would have expected `true. X=bolo; X=modena` but obtained only true which is against the interpretation that should be given to the prolog predicates.

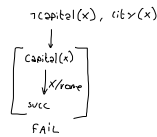
Moreover, while NAF works well for ground terms (terms without variables), its behavior becomes problematic with unbounded (free) variables. These issues arise because Prolog interprets negation

procedurally rather than declaratively. As we have seen at lesson:

```
% Knowledge Base
capital(rome).
region_capital(bolo).
city(X) :- capital(X).
city(X) :- region_capital(X).
```

Given the same KB, we'll notice that by entering the same query in different order the output will change drastically.

```
?- \+capital(X), city(X).
false.
```



The first atom is proven by trying to prove `capital(X)` which succeeds with `X/rome`, so the atom is false and it doesn't proceed with `city/1`.

```
?- city(X), \+capital(X).
X = bolo.
```

The first atom succeeds for `X/rome` and `X/bolo`, then it is evaluated `\+capital(rome)` which fails and `\+capital(bolo)` which succeeds and so we got the result.

6. Introduce the meta-predicate `clause/2`, showing its use by means of a short example program.

The meta-predicate `clause(H, B)` is true if Prolog can unify `H` and `B` respectively with the Head and the Body of a clause stored within the database program.

1. `H` cannot be a variable (but can be a term containing variables), this means that we can't directly access all the heads with a given body.
2. `B` can be a variable or a term.

It is used for example in the vanilla meta-interpreter when dealing with a single atom, since it allows to extract its body so that it can solve it to solve the atom.

A quick example of its use can be also:

```
% KB
p(a) :- g(a).
p(b) :- k(b).

?- clause(p(a), B).
B = g(a).
```

```
?- clause(p(X), B).
```

```
X = a,
```

```
B = g(a) ;
```

```
X = b,
```

```
B = k(b).
```

```
?- clause(H, g(a)).
```

```
ERROR: Arguments are not sufficiently instantiated
```

7. Describe the (operational) semantics of the predicates setof, bagof, and findall. Moreover, the description should be illustrated by some short examples.

Those are predicates that are used in Prolog for collecting solutions to queries into a list.

- **setof(+X, +P, -L)** unifies L with all the instances of X that satisfies P. **If none satisfies P, it fails.** Moreover it also sorts the list in lexicographic order.
- **bagof(+X, +P, -L)** unifies L with all the instances of X that satisfies P, with repetitions. **If none satisfies P, it fails.**
- **findall(+X, +P, -L)** unifies L with all the instances of X that satisfies P, with repetitions. **If none satisfies P, unifies L with an empty list.** Moreover it **does not partition the results based on any free variables in the predicate P.** This means that all possible solutions to the query are collected into a single list, without considering or grouping them by different bindings of free variables.

```
% KB
```

```
parent(john, mary).
```

```
parent(john, mary).
```

```
parent(john, alex).
```

```
parent(alex, sara).
```

```
% Queries
```

```
?- setof(Child, parent(john, Child), Children).
```

```
Children = [alex, mary].
```

```
?- bagof(Child, parent(john, Child), Children).
```

```
Children = [mary, mary, alex].
```

```
?- findall(Child, parent(john, Child), Children).
```

```
Children = [mary, mary, alex].
```

```
?- setof(Child, parent(Parent, Child), Children).
```

```
Parent = alex,
```

```
Children = [sara] ;
```

```
Parent = john,
```

```
Children = [alex, mary].
```

```
?- bagof(Child, parent(Parent, Child), Children).
```

```
Parent = alex,
```

```
Children = [sara] ;
```

```
Parent = john,
```

```
Children = [mary, mary, alex].
```

Have duplicates and are not sorted

Results are partitioned by Parent

```
?- findall(Child, parent(Parent, Child), Children).
Children = [mary, mary, alex, sara].
```

8. Describe the distribution semantics adopted in LPAD, using also a short program to illustrate such semantics.

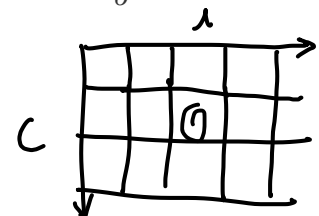
LPAD is a probabilistic logic programming language and so defines a probability distribution over normal logic programs. In particular LPAD is very similar to prolog but with a significant difference: the head of the clauses are extended with disjunctions, and each disjunct is annotated with a probability of its disjunct to occur.

Atoms are assumed to be independent

```
% General structure of the clause(the null case is optional)
outcome1:prob1; ... ; outcomeN:probN; null:prob_no_outcome :- body.
```

Each combination of these possible outcomes identifies a possible world, which as a fact is a standard deterministic logic program where all probabilistic choices have been resolved. We will identify a world with a selection σ , which is the set of all the choices that have been resolved (all the clauses are ground). The choice in the selection σ will be represented as the triple (C, θ, i) , meaning that the world W_σ has in the clause C the substitution θ to the i -th head atom. The probability of that world W_σ is:

$$P(W_\sigma) = P(\sigma) = \prod_{(C, \theta, i) \in \sigma} P(C, i)$$



An example of the semantics is:

```
% KB
sneezing(bob):0.7 ; null:0.3 :- flu(bob).
sneezing(bob):0.8 ; null:0.2 :- hay_fever(bob).
flu(bob).
hay_fever(bob).
```

```
% The world we select
sneezing(bob) :- flu(bob).      Only one world over the 4 possible that we consider for queries
null :- hay_fever(bob).
```

```
% Probability of this world will be (0.7 * 0.2)
```

NB: p(sneezing) is the marginal probability = sum of sneezing in all worlds where sneezing is true

9. Introduce the notions of close world assumption and open world assumption, and to briefly discuss how Prolog and Description Logics deal with these aspects.

In **CWA** (Close World Assumption) we have the principle that **any statement that is not known to be true is assumed to be false** (*absence of evidence is treated as evidence of absence*). So we assume to have a complete and static knowledge base.

In **OWA** (Open World Assumption) we assume that **if the value of a predicate cannot be inferred then it is unknown**. So we assume an incomplete or evolving knowledge base.

Prolog is based on a subset of the CWA, the NaF (Negation as Failure) which makes a local assumption for a specific query: if a fact cannot be derived from the knowledge base in finite time, it is considered false for the purposes of the query. Those 2 approaches differ in the way they handle the absence of information in a knowledge base, since NaF applies to logical reasoning within a proof process, not a global assumption about the domain. **Prolog uses NaF and not CWA because it's not reasonable to assume a complete knowledge**

On the other hand, Description Logic is based on OWA. In order to deal with the non-monotonic reasoning, where new information can alter previous assumptions, if we can't infer a value then it will be unknown until further information will be provided. Before doing that, as we have seen in the **Oedipus paradox**, when it seems that the **KB does not entail the sentence we start by enumerating all the possible worlds by giving a value to the unknowns** and if we get an unique result in all the worlds than we infer that result otherwise will be given the unknown result. Note that the worlds splitting approach is not feasible in CWA because of course we assume that the world we are considering is the only one possible.

10. Briefly introduce the ALC Description Logics, mentioning the operators that are supported (negation, AND, ALL, EXISTS), and their meaning (possibly with a short example for each operator).

ALC (Attributive Language with Complement) is a decidable fragment of Description Logics (DL) used for representing knowledge about concepts and relationships. DL are **based on concepts (categories), roles (relationships) and constants (objects)**. In order to build complex concepts starting from the atomic ones, some of the supported operators are:

1. Negation ($\neg$)

Represents the complement of a concept. If a concept **A** is interpreted as the set of all elements satisfying **A**, then $\neg A$ is the set of all the elements that do not satisfy **A**.

$\Rightarrow \neg \text{Student}$ is the set of individuals who are not students.

2. [ALL r d]

Concept that specifies that all those individuals that are **r**-related only to individuals of class **d**.

$\Rightarrow [\text{ALL :HaveFriends Male}]$ is the set of all the individuals that have 0 or more male friends.

3. [EXISTS n r]

Individuals in the domain that are r -related to at least n other individuals.

$\Rightarrow$ [EXIST 1 :HaveFriends] is the set with all the individual with at least 1 friend.

4. [FILLS r c]

Individuals that are related r -related to the individual identified by c .

$\Rightarrow$ [FILLS :hasFriend Jhon] is the set of all individuals friends of Jhon

5. [AND $c_1 \dots c_n$]

Concept that represents the intersection of multiple concepts, an individual satisfies it if it belongs to all the concepts $c_1, \dots, c_n$.

$\Rightarrow$ [AND Student Male] is the set of all the male students.

11. Within the terminological approach towards the representation of concepts/categories and individuals/instances, the candidate is invited to illustrate the notions of:

- **Disjointness over a set S of categories** ($S = c_1, \dots, c_n$ where $c_1, \dots, c_n$ are categories)
- **Exhaustive Decomposition of a category c into a set S of categories.**
- **Partition of a category c into a set S of categories.**

The candidate is invited to illustrate these notions through a simple example.

In upper ontologies, disjointness, exhaustive decomposition, and partition are introduced to model the relationships between categories (concepts) in a more precise and structured way, using their subclasses relations.

1. **Disjointness** (intrinsic property)

A set of categories S is disjoint iff each category c_i in S is different, meaning that they don't have common objects, formally:

$$\forall c_i, c_j \in S : (c_i \neq c_j) \rightarrow (c_i \cap c_j = \emptyset)$$

Example: ~~$S = \{\text{Dogs}, \text{Cats}\}$~~

$S = \{\text{Mammals}, \text{Birds}\}$

They are disjoint and are not an exhaustive decomposition for Animals for example.

2. **Exhaustive Decomposition** (extrinsic property)

A set of categories S is an exhaustive decomposition of a category c iff each object o belongs to at least a category in S , formally:

$$\forall o \in c : (\exists c_i \in S : o \in c_i)$$

Is not disjoint as amphibians are both water and land animals

Example: ~~$c = \text{NorthAmericans}$~~ and ~~$S = \{\text{Americans, Canadian, Mexicans}\}$~~
 $c = \text{Animal}$ $S = \{\text{LandAnimals, WaterAnimals, FlyAnimals}\}$

3. Partition

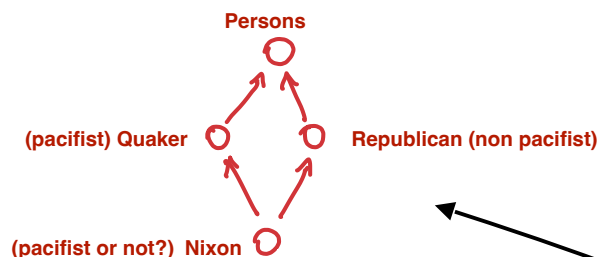
A set of categories S is a partition of a category c iff S is both disjoint and an exhaustive decomposition of c .

Example: $c = \text{Animals}$ and $S = \{\text{Mammals, Birds, Reptiles, Fish, Amphibians}\}$

12. Briefly introduce the notion of Semantic Networks, and to highlight some of the limits that were present in their original formulation.

A semantic network is a structure used to represent knowledge in the form of a directed graph. It consists of vertices, which represent categories and objects, and edges, which represent semantic relations between nodes, that can be:

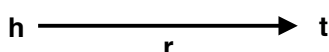
1. Relations between objects
2. Property of a ~~category~~/object
3. Is-a relations **(relation between categories)**
4. Property of the members of a category **(so property of a category)**



Semantic networks are considered a less powerful subset of First-Order Logic (FOL) due to their lack of formal semantics. This trade-off was made to achieve a simpler and more intuitive graphical representation. However, this simplicity limits their expressive power, making semantic networks incapable of representing **negations**, **quantifiers**, **disjunctions**, and **function nestings**. Moreover it has some problems with inheritance since it allows also multiple inheritance that leads to logical inconsistencies.

13. Briefly introduce the Knowledge Graph paradigms, and which are the main differences w.r.t. the Semantic Web proposal.

In a Knowledge Graph (KG), knowledge is represented as a graph structure where terms or entities are the nodes, and the edges represent the relationships between them. They are formally expressed by set of triplets (h, r, t) , where h and t are the head and the tail of the relation (oriented graph) and r is the type of the relation that connects them.



no ontology

- 1) No automated reasoning
- 2) Graph traversal
- 3) Simple vocabulary
- 4) Triplets
- 5) No conceptual schema

vs

- 1) Info inconsistency
- 2) Globality all together
- 3) Info distribution across platforms
- 4) Info incompleteness

It's important to notice that KGs do not enforce a strict conceptual data, so they can integrate heterogeneous data sources with differing semantics, making them more robust in dynamic scenarios. Moreover, queries can be solved by exploring the graph using traditional graph algorithms.

With respect to the Semantic Web proposal instead of reasoning over schemas (T-box) and instances (A-box), KGs treat them as a unified structure containing facts only.

14. Briefly introduce the three different approaches (presented in the course), to deal with the reasoning with temporal information.

1. Propositional logic

We fill how KB with a set of propositions that states the validity of a property at a time t . Then we can implement effect axioms that change the KB over the time simulating actions. This is subject to the frame problem and we could overcome it by implementing frame axioms (For each proposition that is not affected by the action, we will state that it is unaffected). The problem is in the complexity (axiom for each action and each proposition) as it tends to be quadratic.

2. Situation calculus (Green + FOL)

Here we have situations that , that are description of what holds in a given moment. Actions transition the system from one situatio to another, according to their preconditions and their effects. In order to map the changings in the situations over the time we also have fluents, that indicate if a property holds in a given situation. Time is implicit, represented as a sequence of situations.

Action's precondition are defined by axioms:

1. **Possibility axiom:** an action a is possible in a situation s if all its preconditions $\Phi_a(s)$ hold, formally:

$$\Phi_a(s) \Rightarrow \text{Poss}(a, s)$$

2. **Successor state axiom:** A fluent is true in a state t if an action makes it true or does not change if the action does not involve it. Note that solves the frame problem but inefficiently (a clause for each fluent) Formally the axiom is :

$$\text{Poss}(a, s) \Rightarrow \{F(\text{Result}(a, s)) \iff [a = \text{ActionClauseF} \wedge (F(s) \vee a \neq \text{ActionClauseNotF})]\}$$

3. **Unique action axiom:** only one action can be executed in a situation in order to avoid non-determinism and/or conflicts/incoherencies.

3. Event calculus (Kowalski + FOL)

Fluents are reified as terms and a fixed set of terms allows to describe the evolution of the world without the frame problem. The predicates are functions of events E and their impact on fluents F

at/between times T (see predicates at 15). This allows an easier formalism that can be easily implemented in Prolog even though it shows unsafety when fluents or events contains variable because of NaF.

4. Allen's logic of intervals

It is based on intervals as a duration between 2 timepoints i_1, i_2 defined by $\text{Begin}(i_1)$ and $\text{Ends}(i_2)$. There are also other temporal operators as functions of starting/ending points in order to express relations among intervals.

5. Linear Time-Temporal Logic (LTL)

A logic introduces to overcome absolute truth and to represent possibility, necessity, belief, ecc

LTL is a **modal logic** built on atomic propositions, whose values can change along time. It is based on the notion of a world, described in terms of propositions that are true in that world, and on a function that make each world to evolve into a new one.

In order to organize them we have some modal temporal operators:

- **Next** ($\circ\phi$) : A proposition ϕ will be true **<u>in the next moment </u>**.
- **Global** ($\Box\phi$) : A proposition ϕ will be true **<u>always </u>** in the future.
- **Future** ($\Diamond\phi$) : A proposition ϕ will be true **<u>sometimes </u>** in the future.
- **Until** ($\phi U \psi$) : Exists a proposition ψ that is true **<u>in a certain moment </u>** and there exists a proposition ϕ that will be true from now on x **<u>until </u>** ψ .
- **Weak Until** ($\phi W \psi$) : The proposition ϕ **will be true <u>from now on unless </u>** ψ occurs, making ϕ false (but we are not sure that ψ will occur and this is why it is called "weak until").

15. Describe the terminology, ontology, and domain-independent axioms of the Event Calculus Framework, explaining its key predicates, and provide a detailed account of the framework axioms.

In Event calculus we have predicates are functions of events E and their impact on fluents F at/between times T .

The primitive predicates are:

- **HoldsAt**(F, T): The fluent F holds at time T .
- **Happens**(E, T): The event E happened at time T .
- **Initiates**(E, F, T): The event E caused the fluent F to hold at time T .
- **Terminates**(E, F, T): The event E caused the fluent F not to hold at time T .
- **Clipped**(T_1, F, T_2): The fluent F has been made false between T_1 and T_2 .
- **Initially**(F): The fluent F holds at the beginningn (time 0).

The primitive predicates are then combined to define axioms:

- **Domain Independent:** which defines 3 general axioms on how to make Event Calculus work correctly.
 1. A fluent F holds in a certain time T if it was initiated before T and was never clipped since then. $\text{HoldsAt}(F,T) \Leftarrow (\text{Happens}(E,T_1) \wedge \text{Initiates}(E,F,T_1) \wedge (T_1 < T) \wedge \neg \text{Clipped}(T_1,F,T))$
 2. Same as before but with $T = 0$. $\text{HoldsAt}(F,T) \Leftarrow (\text{Initially}(F) \wedge \neg \text{Clipped}(0,F,T))$
 3. A fluent F has been clipped in $[T_1, T_2]$ if happened an event that terminated it. $\text{Clipped}(T_1,F,T_2) \Leftarrow (\text{Happens}(E,T) \wedge (T_1 < T < T_2) \wedge \text{Terminates}(E,F,T))$
 - **Domain Dependent :** which typically use initiates, terminates and clipped to define specific axioms for the domain of interest.
-

16. Explain Forward reasoning in rule-based systems, and highlight the difference w.r.t. backward reasoning.

A rule-based system contains a set of rules in the form of logical implications and facts that can make the rules true. The rules have a structure of $(p_1, \dots, p_n) \rightarrow (q_1, \dots, q_m)$ where on the left-hand-side (LHS) we have the premises and on the right-hand-side (RHS) we have the conclusions. Following *Modus Ponens*, if we have a valid implication and its premises are valid, then its conclusions are valid too.

When a new fact is added to the knowledge base, a rule-based system runs the following steps until the goal is reached or no more rules can be applied:

1. **Rule matching:** search for the rules whose LHS matches the fact and decide which ones will trigger.
2. **Conflict resolution:** triggered rules are put into the Agenda (a queue), and possible conflicts are solved in a predefined order (es. FIFO).
3. **Rule execution:** The RHS of the selected rules are executed and the KB is updated.

Differently from forward reasoning, in backward reasoning the result is obtained starting from the goal by searching for a proof that finds the facts that make the conclusion true. If new facts are added to the knowledge base, the reasoning mechanism need to be restarted as it is unable to dynamically consider changes.

17. Introduce the RETE algorithm, and to show with a short example in natural language the differences between inter-elements and intra-elements patterns/features.

RETE is an efficient algorithm for rule-based systems that implements the matching part of forward reasoning process, so that allows to associate to each rule the facts that matches its LHS.

In order not to iterate over all the facts in the working memory, RETE implements a "conflict set" which stores, for each pattern (conjunct in LHS), what are the facts that match it and then we simply keep that set updated.

Before doing that, in order to make the data more manageable, efficient and scalable, RETE saves/compiles LHS into networks of nodes.

We have then to distinguish between at least 2 types of patterns in the premises:

1. **Intra-elements features**, the features that can be checked within the fact.

es: "If a person is older than 18, then they are an adult." only relies on the age property of a person that could be resolved by the single fact (Person, Mario, 17)

2. **Inter-elements features**, the features that involve more facts.

es: "If a parent of my parent is a Man then he's my grandfather" relies in more than one fact like (Person, Mario, M, child=Piero), (Person, Pjero, M, child=Me)

After that, we finally compile the patterns into the networks:

1. **α -networks**: capture intra-element features and save the results into alpha-memories that can be used by beta-networks.
2. **β -networks**: capture inter-element features thanks to α -networks and save the results into beta-memories, which corresponds to the conflict set.

So the order of the algorithm is:

1. Facts into working memory

2. Matching

1. α -network
2. intra-elements features
3. α -memories
4. β -network
5. inter-elements features
6. β -memories

3. Conflict resolution

Matched rules are put into the Agenda (a queue), and possible conflicts are solved in a predefined order (es. FIFO).

4. Execution

All the rules are executed in a cycle. **There can be many variants**

